Backend Service Detailed Documentation

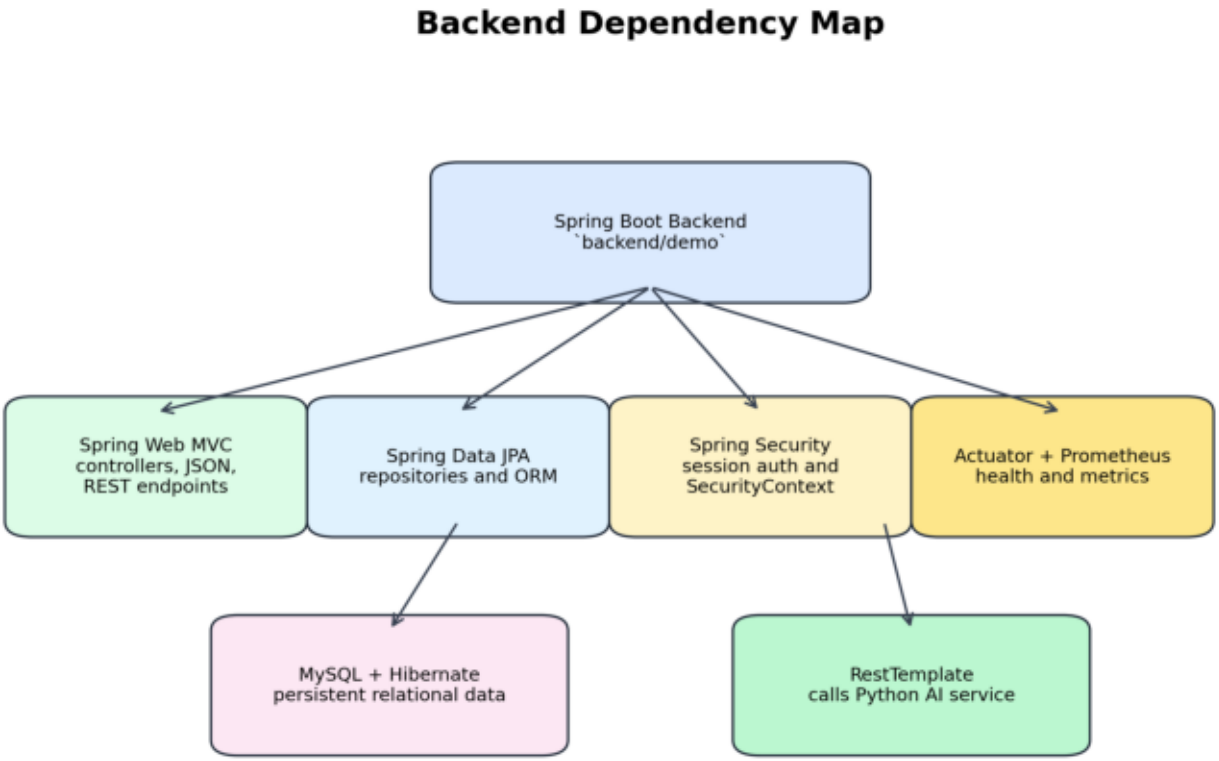
Deep dive into the Java Spring Boot backend in `backend/demo`

- explains the backend from top to bottom
- covers Spring libraries and why each one is present
- documents entities, repositories, services, controllers, and configs
- includes diagrams plus interview-ready talking points

Important note

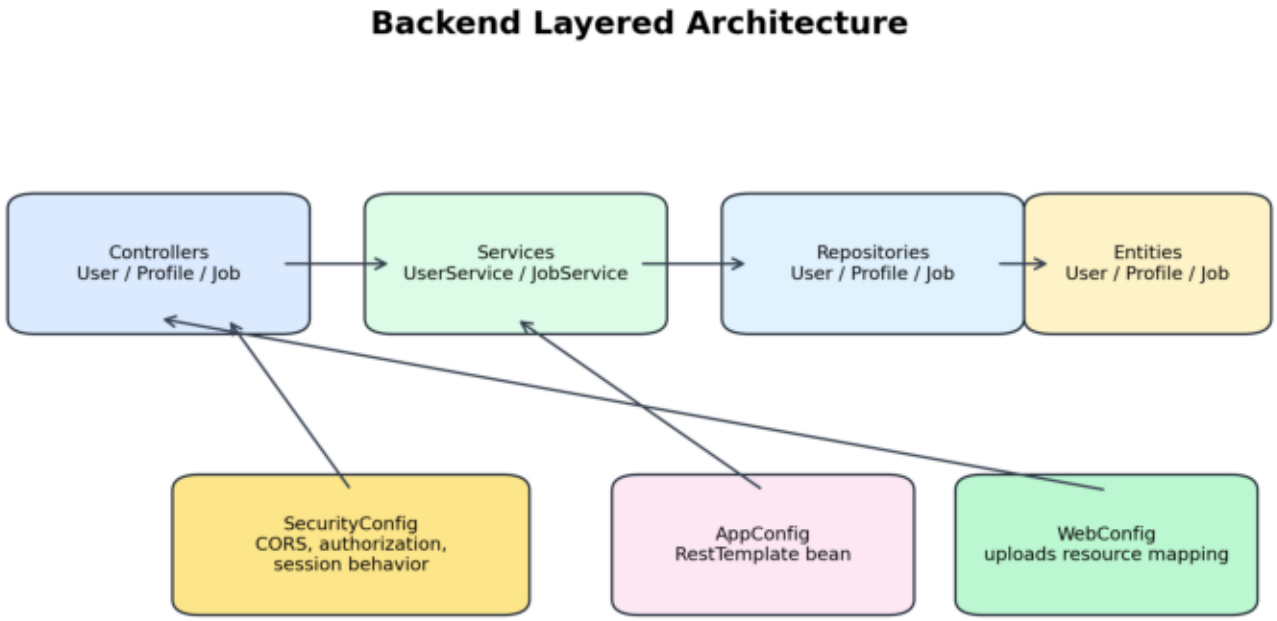
The backend folder contains Java, not Python. This PDF documents the actual backend code present in

Dependency Map



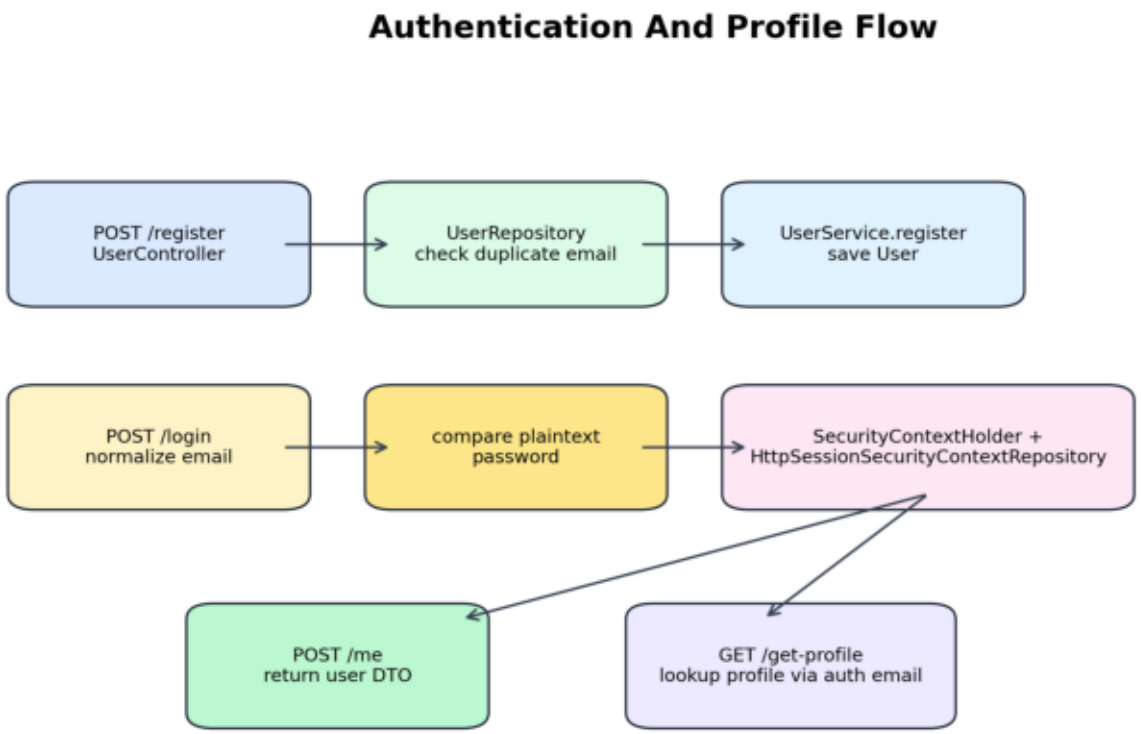
This diagram groups the main backend technologies by responsibility: web API, persistence, security, observability, and Python-service integration.

Layered Architecture



This shows how requests typically move through controllers, services, repositories, entities, and supporting configuration classes.

Authentication Flow



This summarizes the registration, login, session creation, current-user lookup, and profile retrieval flow used by the backend.

Clarification

The `backend` folder does not contain Python code. It contains a Java Spring Boot service under `backend/demo`.

So this document is a deep technical explanation of the backend Java code, its Spring libraries, how the classes are wired together, and how the backend participates in the larger system.

Backend Overview

This backend is the transactional service for the internship platform. Its responsibilities are:
- registering users
- logging users in and out
- restoring the current session user
- creating and reading profile data
- accepting job creation requests
- calling the Python AI service for resume analysis and job-query generation
- exposing health and metrics endpoints through Spring Actuator

Architecturally, it is a monolithic Spring Boot application with a standard layered structure:
- controllers for HTTP endpoints
- services for business logic and outbound service calls
- repositories for persistence
- entities for database mapping
- DTOs for transport
- config classes for beans, security, and static resource handling

It runs on port `8081` and uses MySQL as its database.

Project Configuration Files

`pom.xml` is the Maven build descriptor. It defines:
- project coordinates `com.app:demo:0.0.1-SNAPSHOT`
- parent `spring-boot-starter-parent` version `4.0.3`
- Java version `21`

The main runtime dependencies are:
- `spring-boot-starter-webmvc`: builds REST APIs, request mapping, JSON conversion, and MVC infrastructure
- `spring-boot-starter-data-jpa`: ORM and repository support
- `mysql-connector-j`: MySQL JDBC driver
- `spring-boot-starter-security`: security filter chain, authentication context, and authorization rules
- `spring-boot-starter-actuator`: operational endpoints like health and info
- `micrometer-registry-prometheus`: Prometheus metrics export
- `lombok`: reduces boilerplate, though in this code it is only partially used

Test dependencies:
- `spring-boot-starter-data-jpa-test`
- `spring-boot-starter-webmvc-test`
- `spring-boot-starter-security-test`

Build plugins:
- `maven-compiler-plugin` with Lombok annotation processing
- `spring-boot-maven-plugin` for packaging

`application.properties` sets:
- application name `demo`
- server port `8081`
- MySQL connection to database `Internship`
- username and password both `root`
- Hibernate `ddl-auto=update`
- SQL logging enabled
- all actuator endpoints exposed
- Prometheus endpoint enabled

What The Spring Libraries Do

Spring Boot:
- auto-configures the application based on the classpath and configuration
- starts the embedded server
- wires beans together using dependency injection

Spring Web MVC:
- powers `@RestController`, `@PostMapping`, `@GetMapping`, `ResponseEntity`, and request-body binding
- converts JSON request bodies into Java objects and Java objects back into JSON responses

Spring Data JPA:
- powers `JpaRepository`
- eliminates boilerplate CRUD code
- connects entity classes to relational tables through Hibernate

Hibernate:
- is the default JPA provider
- translates entity operations into SQL
- handles schema updates because `ddl-auto=update` is enabled

Spring Security:
- inserts a security filter chain in front of requests
- stores authentication in `SecurityContextHolder`
- helps support session-based login

Spring Actuator and Micrometer:
- expose backend operational data
- provide health and metrics endpoints
- export Prometheus-compatible metrics

Lombok:
- `@Data` on `User` would normally generate getters, setters, `equals`, `hashCode`, and `toString`
- but the class also includes manual getters and setters, so Lombok is not adding much value there

What The Spring Libraries Do

RestTemplate:
- is Spring’s classic synchronous HTTP client
- used here to call the Python service from the Java backend

Entry Point: DemoApplication

`DemoApplication.java` is the backend entry point.

Important pieces:
- `@SpringBootApplication` marks the main class and combines component scanning, auto-configuration, and extra configuration support.
- `SpringApplication.run(DemoApplication.class, args)` boots the application context and starts the server.

Why this class exists:
- It tells Spring where to start scanning for components.
- Because it sits at package `com.app.demo`, Spring will scan nested packages like `Config`, `Controller`, `Service`, `Repository`, and `Entity`.

This class is intentionally minimal because Spring Boot encourages convention over configuration.

Configuration Classes

`AppConfig.java`
- annotated with `@Configuration`
- defines a `@Bean` method returning a `RestTemplate`
- purpose: make a reusable HTTP client available for dependency injection

`SecurityConfig.java`
- annotated with `@Configuration`
- exposes a `SecurityFilterChain`
- disables CSRF
- permits actuator routes
- permits `/login`, `/register`, `/create-job`, `/me`, `/add-profile`, and `/upload`
- then also marks `/me` as authenticated
- disables form login
- defines a `CorsConfigurationSource` allowing `http://localhost:5173`

Important subtlety:
The authorization rules are ordered. Since `/me` appears in the earlier `permitAll()` matcher, the later authenticated matcher is effectively redundant. The controller still performs auth checks itself, but the route-level security declaration is looser than intended.

`WebConfig.java`
- implements `WebMvcConfigurer`
- maps `/uploads/**` to the local filesystem `uploads` directory
- purpose: if the app stores files there, they can be served back as static resources

Interesting note:
The current code does not actually save uploaded files into that directory, because file upload is proxied to the Python service instead.

Entity Classes And Database Modeling

`User`
- entity mapped to table `users`
- fields: `id`, `firstName`, `lastName`, `email`, `password`
- `email` is unique and non-null
- one-to-one relationship with `Profile`

`Profile`
- fields: `id`, `phoneNumber`, `gender`, `state`
- owns the one-to-one relationship using `@JoinColumn(name = "user_id", nullable = false)`
- each profile belongs to exactly one user

`Job`
- fields: `id`, `title`, `company`, `location`, `skills`
- intended to represent job records that can later be semantically indexed

Important modeling caveats:
- `Profile.id` is `Long`, but `ProfileRepository` uses `Integer` as the repository ID type
- `Job.skills` is a `List<String>` but lacks JPA collection mapping annotations such as `@ElementCollection`

Why these matter:
- repository generic mismatches can cause confusion or type-safety issues
- unannotated collections in JPA are not a complete relational mapping and would normally need extra configuration

Repository Layer

Repositories inherit from `JpaRepository`, which means Spring automatically generates CRUD implementations at runtime.

`UserRepository`
- extends `JpaRepository<User, Integer>`
- adds `findByEmail(String email)`
- Spring derives the SQL from the method name

`ProfileRepository`
- extends `JpaRepository<Profile, Integer>`
- adds `getProfileByUserId(Integer id)`
- Spring interprets this as a query across the `user` relation and its `id`

`JobRepository`
- extends `JpaRepository<Job, Integer>`
- currently uses only inherited CRUD methods

Why repositories are useful:
- they isolate persistence concerns from controllers
- they reduce boilerplate
- they let the service layer work in terms of domain objects rather than handwritten SQL

DTO Classes

The DTO package defines transport models so the API does not always expose entities directly.

`UserRegisterDTO`
- incoming payload for registration
- fields: `id`, `firstName`, `lastName`, `email`, `password`

`UserResponseDTO`
- safe outward-facing user projection
- fields: `id`, `firstName`, `lastName`, `email`

`LoginRequestDTO`
- incoming login payload
- fields: `email`, `password`

`LoginResponseDTO`
- outgoing login status
- fields: `message`, `success`

`ProfileRequestDTO`
- incoming profile creation payload
- fields: `phoneNumber`, `gender`, `state`

`JobRequestDTO`
- payload sent from the Java backend to the Python AI service
- fields: `id`, `title`, `company`, `location`, `skills`

`PythonResponseDTO`
- Java-side representation of the Python service response
- fields include generated query, embedding length, and message

`JobResponseDTO`
- final response returned by the backend after saving a job and contacting Python

Why DTOs matter:

DTO Classes

- they control shape and intent
- they prevent accidental overexposure of entity internals
- they let the backend maintain separate contracts for frontend traffic and service-to-service traffic

Service Layer

`UserService`
- annotated with `@Service`
- has two main responsibilities: register users and forward PDFs to FastAPI

`register(UserRegisterDTO userRegisterDTO)`
- creates a new `User`
- copies fields from the DTO
- normalizes email to lowercase and trims it
- stores plaintext password directly
- saves the entity via `UserRepository`
- maps the saved user into `UserResponseDTO`

`sendPdfToFastApi(MultipartFile file)`
- builds a multipart/form-data request manually
- wraps uploaded bytes in `ByteArrayResource`
- posts to `http://localhost:8000/analyze-resume`
- returns the Python JSON as a raw `Map<String, Object>`

Important design note:
`UserService` constructs its own `RestTemplate` instead of using the `RestTemplate` bean from `AppConfig`, so there are two creation styles in the codebase.

`JobService`
- annotated with `@Service`
- injected with `JobRepository` and the shared `RestTemplate` bean
- owns job persistence plus semantic indexing orchestration

`saveJobAndGenerateQuery(Job job)`
- saves the `Job` first
- builds a `JobRequestDTO`
- calls `http://127.0.0.1:8000/generate-query`
- reads the Python response into `PythonResponseDTO`
- builds a `JobResponseDTO` for the caller

Service Layer

Consistency consideration:
If the Python call fails after the database save succeeds, the job exists in MySQL but not in Pinecone. So persistence and semantic indexing are not transactional together.

`FileStorageService`
- currently contains only a comment `// Removed.`
- this indicates either a deleted feature or an abandoned placeholder

Controller Layer

`UserController`
- `@RestController`
- `@CrossOrigin` allows the frontend origin with credentials
- owns `/register`, `/logout`, `/login`, `/me`, and `/get-profile`

`POST /register`
- checks whether the email already exists
- lowercases and trims email
- delegates to `UserService.register`
- returns `400` for duplicate email

`POST /logout`
- clears `SecurityContextHolder`
- invalidates the current session
- returns a plain success string

`POST /login`
- lowercases and trims email
- loads user by email
- compares plaintext password directly
- if valid, creates `UsernamePasswordAuthenticationToken`
- stores auth in the session via `HttpSessionSecurityContextRepository`

`POST /me`
- reads the current `Authentication`
- rejects unauthenticated or anonymous access
- returns the current user as `UserResponseDTO`

`GET /get-profile`
- reads the current user email from the security context
- loads the user and then the profile
- returns `404` if either is absent

`ProfileController`

Controller Layer

- owns `/upload` and `/add-profile`

`POST /upload`
- accepts a multipart file
- delegates to `UserService.sendPdfToFastApi`
- returns proxied JSON from Python

`POST /add-profile`
- manually checks authentication
- loads the authenticated user
- prevents duplicate profile creation
- creates and saves a `Profile`

`JobController`
- minimal controller with one endpoint
- `POST /create-job` forwards the incoming `Job` to `JobService.saveJobAndGenerateQuery`

Authentication Design

This backend uses session-style authentication rather than JWT.

How it works in this code:
1. The login endpoint validates email and password.
2. It creates a `UsernamePasswordAuthenticationToken`.
3. It stores the authentication in `SecurityContextHolder`.
4. It saves the context using `HttpSessionSecurityContextRepository`.
5. Future requests carrying the session cookie can recover the authenticated principal.

Why `SecurityContextHolder` matters:
- it is Spring Security’s central thread-local store for the current request’s authentication state

Why `HttpSessionSecurityContextRepository` matters:
- it persists the security context into the server-side HTTP session

Important security caveats:
- passwords are stored unhashed
- there is no `UserDetailsService`
- no password encoder is configured
- CSRF is disabled globally
- some endpoints rely on manual controller checks because the route security is not fully aligned with intent

External Integration With The Python Service

This backend integrates with the Python AI service in two places.

Resume analysis path:
- `POST /upload` in `ProfileController`
- calls `UserService.sendPdfToFastApi`
- Java builds a multipart request and forwards the file to `http://localhost:8000/analyze-resume`

Job indexing path:
- `POST /create-job` in `JobController`
- calls `JobService.saveJobAndGenerateQuery`
- Java sends JSON to `http://127.0.0.1:8000/generate-query`

Why the backend talks to Python:
- the Java service is optimized for CRUD, sessions, and relational persistence
- the Python service is optimized for resume parsing, embeddings, and vector search

This is a practical split:
- MySQL remains the source of truth for transactional records
- Pinecone remains the source of truth for semantic representations

Testing And Observability

Testing:
- there is one test class, `DemoApplicationTests`
- it contains only `contextLoads()`
- this verifies that the Spring application context can start

What is missing:
- controller tests
- service tests
- repository tests
- security behavior tests
- integration tests for Python-service calls

Observability:
- actuator endpoints are exposed
- Prometheus metrics are enabled

What this gives you:
- health endpoint for uptime checks
- metrics export for dashboards or scraping

What is not present:
- custom metrics
- structured logging strategy
- distributed tracing

Strengths Of The Backend

Good parts of the current backend:
- clear layered organization
- reasonable separation between controllers, services, repositories, and DTOs
- easy-to-follow request flow
- working cross-service integration with the Python AI layer
- actuator and metrics support are already present
- email normalization is handled during login and registration
- one-to-one profile modeling is straightforward

For interview discussion, these strengths show:
- knowledge of Spring conventions
- awareness of service boundaries
- ability to connect transactional and AI-oriented subsystems

Risks And Improvement Ideas

Current risks:
- plaintext password storage and comparison
- permissive or inconsistent security matcher configuration
- incomplete JPA mapping for `Job.skills`
- repository generic mismatch for `Profile`
- duplicate `micrometer-registry-prometheus` dependency in `pom.xml`
- `UserService` bypasses the shared `RestTemplate` bean
- no resilience around outbound HTTP calls
- almost no tests

Improvements that would help:
- add `BCryptPasswordEncoder`
- use a proper authentication service or `UserDetailsService`
- align security matcher rules with controller intent
- annotate `Job.skills` correctly for JPA
- add exception handlers with `@ControllerAdvice`
- centralize outbound HTTP configuration, timeouts, and error handling
- expand automated tests across controller, service, and integration levels
- move hardcoded service URLs into configuration properties

Interview Explanation Script

If you need to explain this backend in an interview, say:

1. The backend is a Spring Boot service that owns transactional data and session-based authentication.
2. It persists users, profiles, and jobs in MySQL through Spring Data JPA and Hibernate.
3. Controllers accept frontend requests, services implement business logic, and repositories handle persistence.
4. Spring Security is used to manage session authentication through `SecurityContextHolder` and the HTTP session.
5. When a job is created or a resume is uploaded, the backend calls a separate Python service through `RestTemplate`.
6. That Python service handles AI-specific tasks such as resume parsing and semantic indexing.

If they ask what you would improve first:
Say password hashing, stronger security rules, proper JPA mapping for job skills, and broader test coverage.